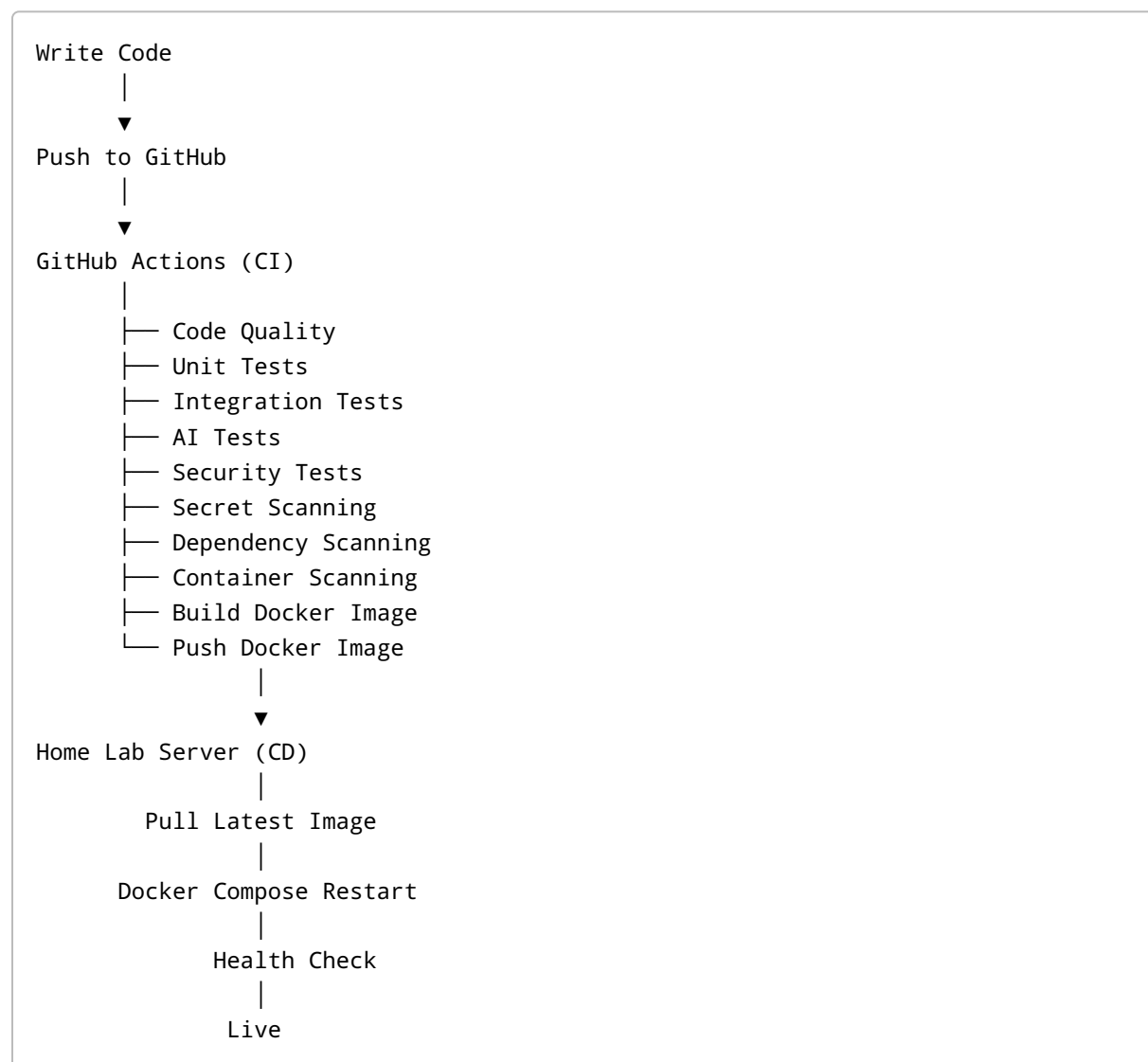


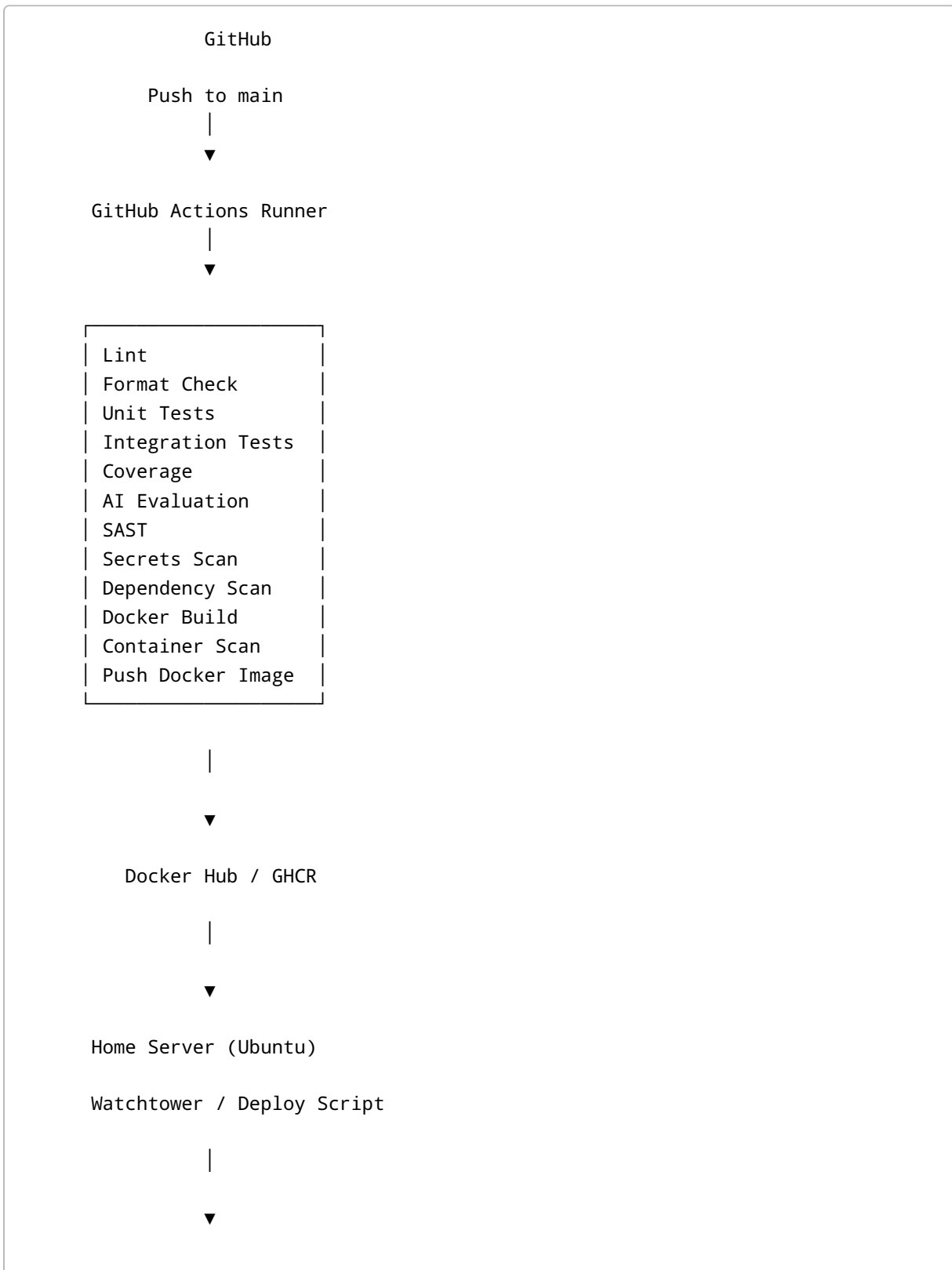
Production CI/CD Pipeline for a LangChain + LangGraph AI Application

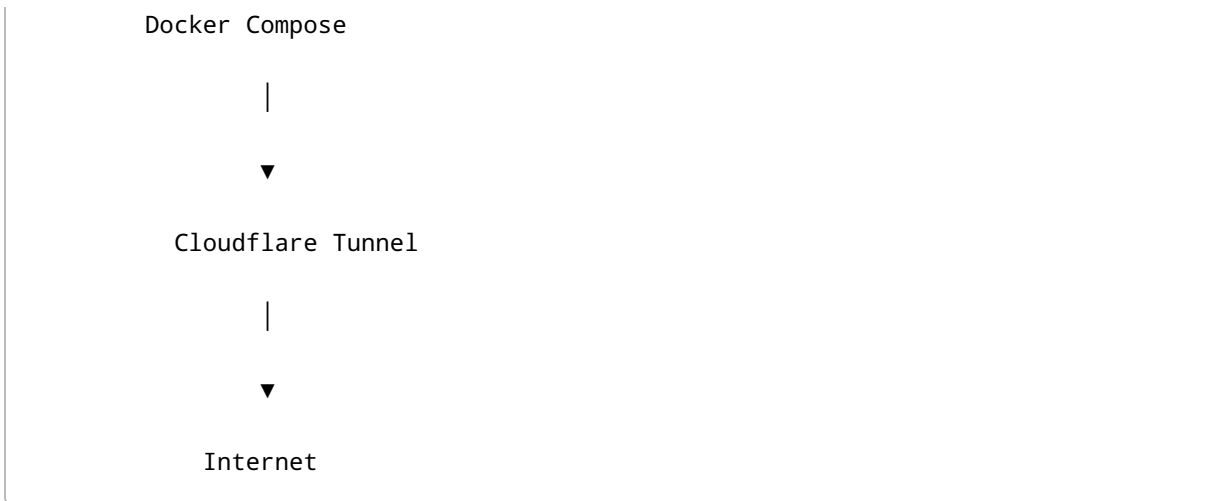
Given everything you've shared over the past few days (Ubuntu homelab, Cloudflare Tunnel, Docker, LangGraph/LangChain Python application, preference for open-source software), I would design your CI/CD pipeline almost exactly like a modern SaaS company would, except using mostly free tools.

The biggest thing to understand first is that **CI (Continuous Integration)** and **CD (Continuous Delivery/Deployment)** are two separate stages.



Overall Architecture





Step 1: Source Control

Use GitHub.

Every feature gets its own branch.

```
main
develop
feature/chat-memory
feature/rag
feature/search
```

Step 2: GitHub Actions

GitHub Actions becomes your CI server.

Every push automatically triggers the pipeline.

No manual work is required.

Stage 1: Code Formatting

Tools

```
Black  
isort  
Ruff
```

Checks

- Formatting
 - Import ordering
 - Code style
 - Common programming mistakes
-

Stage 2: Type Checking

Tool

```
mypy
```

Checks

```
Incorrect types  
None errors  
Wrong return values  
Missing attributes  
etc.
```

Stage 3: Unit Testing

Framework

```
pytest
```

Plugins

```
pytest-cov
pytest-xdist
pytest-mock
pytest-asyncio
```

Example Test Files

```
test_vectorstore.py
test_llm.py
test_embedding.py
test_agent.py
test_tools.py
test_api.py
test_database.py
```

These tests run automatically on every push.

Stage 4: Integration Testing

Tests whether different components work together.

Example:

```
FastAPI
  ↓
LangGraph
  ↓
Postgres
  ↓
Redis
  ↓
Gemini API
```

Stage 5: End-to-End Testing

Run the entire application exactly like a user would.

Example:

```
graph TD; A[User asks question] --> B[Agent answers]; B --> C[Memory stored]; C --> D[Memory retrieved]; D --> E[Response generated];
```

Stage 6: AI Testing

Modern AI applications also test the AI itself.

Recommended Tools

- LangSmith (Free Tier)
- DeepEval
- Promptfoo
- Ragas
- OpenEvals

Metrics

- Retrieval correctness
 - Hallucination score
 - Faithfulness
 - Context precision
 - Context recall
 - Answer relevance
 - Tool usage
 - Agent trajectory
 - Latency
 - Cost
-

Stage 7: Code Coverage

Tool

pytest-cov

Target

90%

Deployment Rule

Never deploy if coverage falls below:

80%

Stage 8: Security

Static Application Security Testing (SAST)

Tools

Semgrep
Bandit

Detects

- SQL Injection
- Command Injection
- Unsafe pickle
- eval()
- exec()
- Hardcoded passwords
- Many other insecure coding patterns

Secret Scanning

Tool

Gitleaks

Detects:

- API Keys
 - Gemini Keys
 - OpenAI Keys
 - JWT Secrets
 - AWS Credentials
 - Passwords
 - Tokens
-

Dependency Vulnerability Scanning

Tools

pip-audit
Safety
Dependabot

Checks for:

- Known CVEs
 - Vulnerable packages
 - Outdated dependencies
-

SBOM Generation

Tool

Syft

Creates a Software Bill of Materials containing every dependency and version.

Container Vulnerability Scanning

Tool

Trivy

Scans:

- Linux packages
- Python packages
- OpenSSL
- glibc
- Container CVEs

Container Best Practices

Trivy also checks:

- Running as root
- File permissions
- Dockerfile mistakes
- Embedded secrets
- Unnecessary packages

Dockerfile Linting

Tool

Hadolint

AI Code Review

Recommended Tools

- GitHub Copilot Code Review
- CodeRabbit
- Reviewdog
- DeepSource
- Qodo

- Bito

Recommended combination:

```
Semgrep  
+  
CodeRabbit  
+  
Reviewdog
```

License Scanning

Tools:

- FOSSA
- ScanCode Toolkit

Useful for ensuring compliance with open-source licenses.

Docker Build

After every test passes:

```
docker build
```

Container Image Signing

Tool

```
Cosign
```

Every image is cryptographically signed to protect against tampering.

Container Registry

Although Docker Hub works well, GitHub Container Registry (GHCR) is recommended because it offers:

- Better GitHub integration
 - No Docker Hub rate limits
 - Good free private image support
-

Deployment

Option 1 — GitHub Actions over SSH

```
GitHub
  ↓
SSH
  ↓
Ubuntu Server
  ↓
docker compose pull
  ↓
docker compose up -d
```

Simple and widely used.

Option 2 — Watchtower (Recommended for Homelab)

Watchtower monitors your container registry.

When a new image appears:

```
Pull Image
  ↓
Stop Old Container
  ↓
Start New Container
```

No SSH is required.

Option 3 — Self-hosted GitHub Runner

Install a GitHub Actions Runner directly on your Ubuntu server.

Very common in production.

Health Checks

After deployment:

```
GET /health
GET /ready
GET /live
```

If the application fails health checks, automatically roll back.

Monitoring

Infrastructure

```
Prometheus
Grafana
Node Exporter
cAdvisor
```

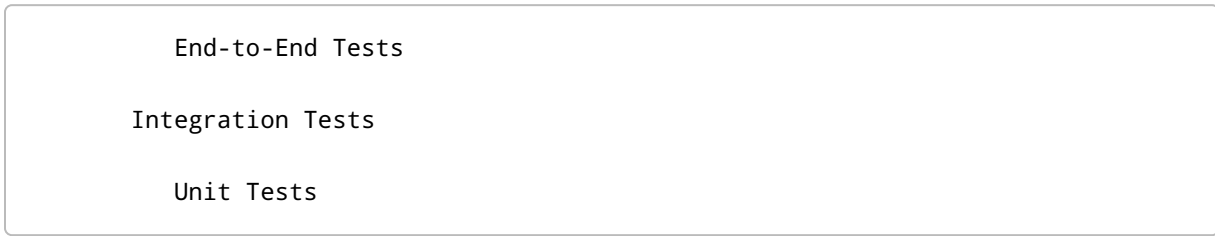
Application

```
OpenTelemetry
LangSmith
Grafana Tempo
Loki
```

Logs

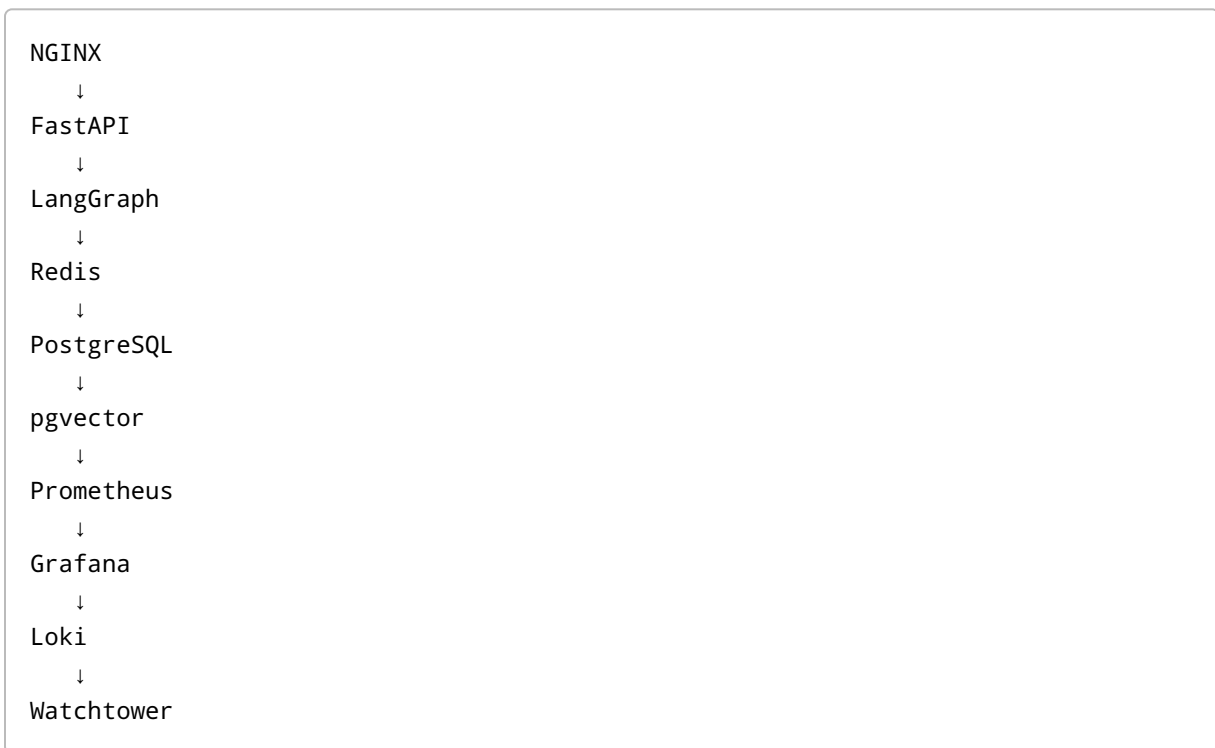
```
Loki
Promtail
```

Testing Pyramid



Most tests should be unit tests because they are the fastest and easiest to maintain.

Docker Compose Stack



Everything runs inside Docker containers.

Firewall Configuration

Because you use a Cloudflare Tunnel:

Do NOT expose:

80
443

Cloudflare Tunnel handles inbound HTTPS traffic.

Also keep these internal:

8000
8080
5000
3000

Only publish them if you explicitly need local LAN access.

SSH

The only port you may expose is:

22

Even then:

- Restrict it to your LAN if possible.
- For remote administration, prefer Cloudflare Zero Trust, WireGuard, or Tailscale instead of exposing SSH directly.

Never expose Postgres, Redis, or pgvector to the internet.

Docker Networks

Separate services into dedicated Docker networks:

```
Proxy Network
  ↓
Application Network
  ↓
Database Network
```

↓
Monitoring Network

Each container joins only the networks it actually requires.

Secret Management

Never store secrets in Git.

Use:

GitHub Secrets
↓
GitHub Actions
↓
Docker Secrets

Avoid embedding secrets inside Docker images or source code.

Recommended Technology Stack

Category	Tool
Git Hosting	GitHub
CI/CD	GitHub Actions
Formatter	Black
Linter	Ruff
Import Sorting	isort
Type Checking	mypy
Unit Testing	pytest
Coverage	pytest-cov
Python Security	Bandit
General Security	Semgrep
Secret Scanning	Gitleaks

Category	Tool
Dependency Scanning	pip-audit
Container Scanning	Trivy
Dockerfile Linting	Hadolint
SBOM	Syft
Image Signing	Cosign
AI Evaluation	DeepEval, Promptfoo, Ragas
AI Code Review	CodeRabbit
Container Registry	GHCR (or Docker Hub)
Deployment	GitHub Actions or Watchtower
Monitoring	Prometheus + Grafana
Logging	Loki + Promtail
Metrics	OpenTelemetry
Reverse Proxy	NGINX
Tunnel	Cloudflare Tunnel
Database	PostgreSQL + pgvector
Cache	Redis

Recommended Learning Path

Instead of implementing everything on day one, build your pipeline in stages:

1. GitHub Actions (linting, formatting, unit tests)
2. Security scanning (Semgrep, Bandit, Gitleaks, pip-audit)
3. Docker image build and Trivy container scanning
4. Push signed images to GHCR or Docker Hub
5. Automatic deployment to your Ubuntu homelab
6. AI evaluation (DeepEval, Promptfoo, Ragas)
7. Monitoring, logging, health checks, and automatic rollback

This mirrors how real engineering teams evolve their CI/CD infrastructure and makes each stage much easier to understand, troubleshoot, and maintain.

Finally, treat your infrastructure as code. Keep your GitHub Actions workflows, Dockerfiles, Docker Compose files, monitoring configuration, and deployment scripts under version control alongside your application so that your complete production environment can be recreated from the repository at any time.